

PZEM

A hardware printer emulator for the VZ200/VZ300 Part 1



Introduction

This article will capture the physical hardware and software design of a minimalist circuit that plugs into the peripheral port of a VZ, to emulate a Centronics printer using an arduino based microcontroller and SD card combo.

Part 2 of the article will include a design for capturing the print data as a bitmap image, reproducing the “dot matrix” appearance of a 1980 era epson printer, and printing directly to a modern printer via wifi.

Following this article a user could choose to:

- Modify an VZ existing printer interface, by cutting the ribbon cable connecting the printer plug and connecting the wires directly to a microcontroller
- Keep the existing printer interface intact, and connect the microcontroller to a printer plug via a matching printer socket
- Build a completely new unit from readily available parts, bypassing the centronics plug/socket completely

History

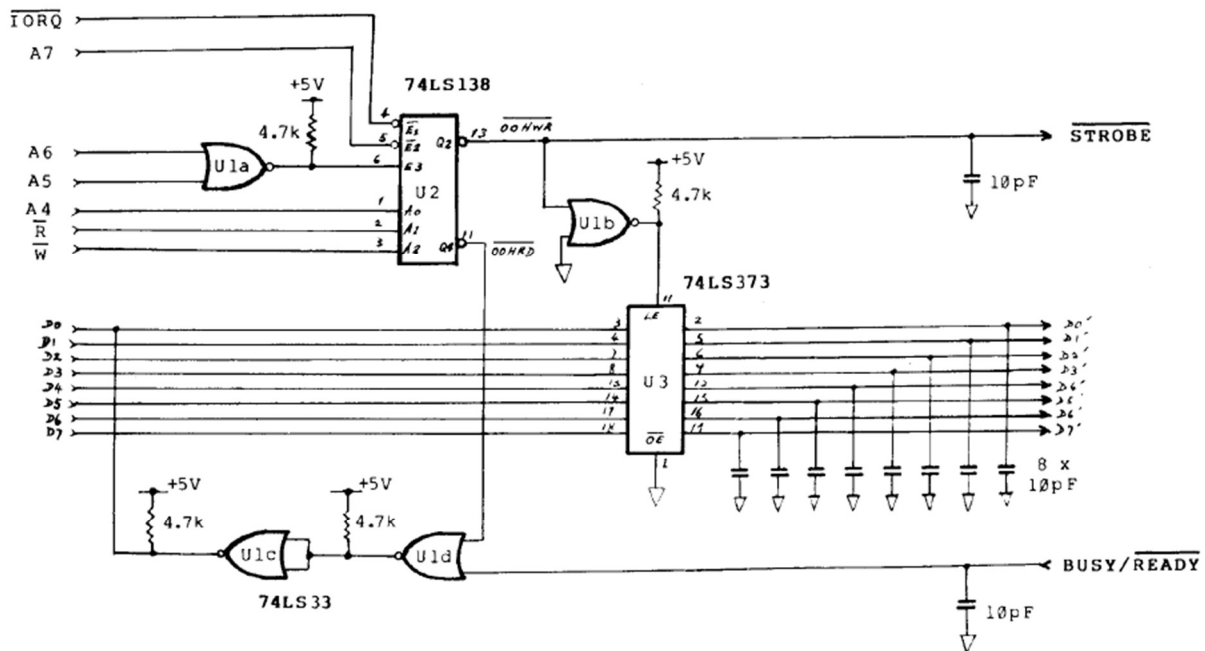
The original 1980s era VZ printer interface plugged into the 30 pin I/O connector at the rear of the machine - the same connector used for the joystick module. The printer interface sent a "strobe" signal to check the status of the connected printer. The printer would respond with a "busy" or "ready" signal. If the printer was in the "ready" state, it could receive the next 8 bits of print data from the VZ. The VZ rom included routines to check the state of the printer and output the print data on a specific port.

Design

The original circuit board contains three ICs:

- 74LS138 address decoder
- 74LS33 NOR buffer
- 74LS373 8 bit latch

The circuit (found in the VZ200 technical reference manual) is copied below



VZ-200 PRINTER INTERFACE

How it works

Address range 00h - 0Fh is reserved for a printer. In binary this is 0000 0000 to 0000 1111. So the lower 4 bits (A0 to A3) can be used for printer communication. By definition this also means bits A4 to A7 must be excluded to not interfere with any other I/O devices connected like the floppy disk controller, joysticks, modem, or memory bank switching.

Address lines A4 to A7 are connected to the input lines of the 74LS138 address decoder. Note that address lines A5 and A6 are fed into a NOR gate, with the output connected to line E3 on the 74LS138. The IORQ, RD and WR signals are fed into the remaining input lines.

Address lines A4 to A7 MUST be low for the 74LS138 to trigger an output. A0 to A3 can be low or high, eg range 0000 to 1111. IORQ must be active low (an I/O instruction is in progress) and /RD or /WR active low to indicate a read or write operation is in progress. This mapping can be displayed using the digram below, using the 74LS138 truth table

Inputs						Outputs							
$\overline{E_1}$	$\overline{E_2}$	E_3	A_0	A_1	A_2	$\overline{0}$	$\overline{1}$	$\overline{2}$	$\overline{3}$	$\overline{4}$	$\overline{5}$	$\overline{6}$	$\overline{7}$
H	X	X	X	X	X	H	H	H	H	H	H	H	H
X	H	X	X	X	X	H	H	H	H	H	H	H	H
X	X	L	X	X	X	H	H	H	H	H	H	H	H
L	L	H	L	L	L	L	H	H	H	H	H	H	H
L	L	H	H	L	L	H	L	H	H	H	H	H	H
L	L	H	L	H	L	H	H	L	H	H	H	H	H
L	L	H	H	H	L	H	H	H	L	H	H	H	H
L	L	H	L	L	H	H	H	H	H	L	H	H	H
L	L	H	H	L	H	H	H	H	H	H	L	H	H
L	L	H	L	H	H	H	H	H	H	H	H	L	H
L	L	H	H	H	H	H	H	H	H	H	H	H	L

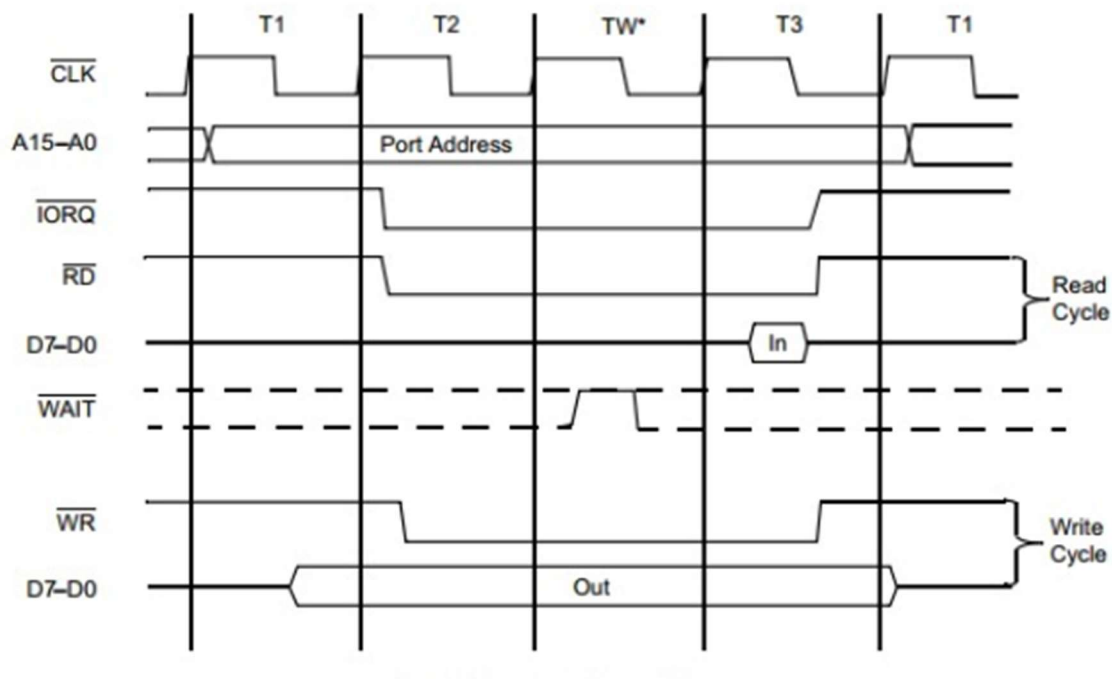
IORQ	E1
A7	E2
! (A6+A5)	E3
A4	A0
/RD	A1
/WR	A2

A /WR operation will bring 74LS138 output O2 to active low

A /RD operation will bring 74LS138 output O4 to active low

Write Operation

Output O2 connects to the printer "strobe" line, which asks the printer if it is ready to receive new data. O2 is also connected to a NOR gate, with the second input tied to ground, eg always LOW. The output of the NOR gate connects to the "latch enable" pin of the 74LS373. The effect of this is the strobe signal will be sent to the printer, and almost immediately after, the contents of the Z80 data bus will be latched on the second half of cycle T3, as soon as IORQ returns to HIGH state. (See diagram below). The output lines of the 74LS373 latch connect directly to the printer.



Read Operation

Output O4 connects to the printer “busy/ready” line, which is the response the printer sends back after receiving a strobe signal. A “busy” response (HIGH) indicates the printer is still printing the last character sent. A “ready” response (LOW) indicates the printer can start printing the last character sent. The printer will then immediately set the busy line to HIGH while it is printing. A CPU can output a block of characters many times faster than a printer can physically print them, and so will be constantly polling the busy line waiting for the printer to be ready.

The “busy/ready” line combined with O2 is fed to another NOR gate (U1d), which is inverted again via U1c. The output of U1c connects to data bit D0 on the Z80 data bus. So a “busy” response will set D0 to HIGH, and a “ready” response will set D0 to LOW. The 74LS33 is a quad 2-input NOR buffer with open collector outputs. This requires a pullup resistor on each of the output lines, but can supply or sink enough current to drive the contents of a data bus (compared to a 74LS02 NOR gate which is only suitable for glue logic)

Replacing the printer with a microcontroller

The 1980s era printers with a centronics port in working condition are rare (noisy too!), as are the consumables. Many of these printers required a stack of continuous paper with perforated edges, that had pin feed holes for the tractor feed to pull the paper through the printer. Modern printers are now connected via usb, or wifi and use a tray of standard A4 paper.

Using a microcontroller as the “printer” involves replicating the strobe/busy signals in software, and writing the received print data to an sd card. The sd card can then be connected to a modern computer, and the contents printed to a modern printer if required.

The microcontroller of choice requires ten available I/O gpio pins. Eight of these pins are for data lines D0 to D7, a strobe pin, and a busy/ready pin.

The original microcontroller used to prototype this was an “Arduino Nano”, chosen because it is cheap and uses 5v logic, which allows it to interface directly with the Z80 and supporting chips without requiring level converters.

I quickly switched to a microcontroller with more RAM, flash storage and built SD card, because I have a few of these spare and it will allow for additional features like print to bitmap.

The microcontroller chosen by the user will require an interrupt enabled pin connected to the strobe line. The arduino nano for example has two interrupt enabled pins, others have more. The pin numbers in the following sketch will need to be adjusted for the microcontroller used.

Logic wise, there is not much to it. The falling edge of the strobe signal will trigger an interrupt, which sets a “printerBusy” flag. The main loop then checks if printerBusy has been set, and if so

1. Sets the busy pin to HIGH for the host computer to check status
2. Reads the 8 data bits from the latch to construct a byte
3. Write the byte to the SD card
4. Set the busy pin to LOW for the host computer to check status

The main loop also keeps track of time elapsed since the last write, to allow a “flush” of any pending writes to the SD card after 5 seconds. This “trick” allows data to be physically written to the card without constantly having to open/close the file, and minimises the risk of data loss if the VZ is turned off or the card removed.

The complete, working sketch is included below

```
#include <SD.h>
#include <SPI.h>

const int ledPin = 13;

#define D0      25
#define D1      26
#define D2      27
#define D3      28
#define D4      29
#define D5      30
```

```

#define D6      31

#define D7      32


volatile bool printerBusy = false;
unsigned long lastFileBufferWrite = 0;


File printerFile;


#define BUSY_PIN 0
#define STROBE_PIN 1 // configured as interrupt pin


FASTRUN void PRINTER_STROBE_ISR()
{
    printerBusy = true;
}


void setup(void)
{

    pinMode(ledPin, OUTPUT);

    if (SD.begin(BUILTIN_SDCARD))
    {
        printerFile = SD.open("printer.txt", FILE_WRITE);
        if (printerFile)
        {
            printerFile.println("sd card found");
        }
    }


    // Data pins connected to 74LS373 latch (printer data)
    pinMode(D0, INPUT);
    pinMode(D1, INPUT);
    pinMode(D2, INPUT);
    pinMode(D3, INPUT);
    pinMode(D4, INPUT);
    pinMode(D5, INPUT);
    pinMode(D6, INPUT);

```

```

pinMode(D7, INPUT);

pinMode(STROBE_PIN, INPUT_PULLUP);
pinMode(BUSY_PIN, OUTPUT);
digitalWriteFast(BUSY_PIN, LOW);

lastFileBufferWrite = millis();
attachInterrupt(digitalPinToInterrupt(STROBE_PIN), PRINTER_STROBE_ISR, FALLING);

}

void loop()
{
    if (printerBusy)
    {
        digitalWriteFast(ledPin, HIGH);    // light up LED

        /*
        *
        *   set busy line, read byte from latch, write byte to sd card
        *   then clear busy line
        *
        */
        digitalWriteFast(BUSY_PIN, HIGH);

        byte b = 0;

        if (digitalReadFast(D0) == HIGH) b |= 1;
        if (digitalReadFast(D1) == HIGH) b |= 2;
        if (digitalReadFast(D2) == HIGH) b |= 4;
        if (digitalReadFast(D3) == HIGH) b |= 8;
        if (digitalReadFast(D4) == HIGH) b |= 16;
        if (digitalReadFast(D5) == HIGH) b |= 32;
        if (digitalReadFast(D6) == HIGH) b |= 64;
        if (digitalReadFast(D7) == HIGH) b |= 128;
    }
}

```

```

printerFile.write(b);

digitalWriteFast(BUSY_PIN, LOW);
digitalWriteFast(ledPin, LOW);
printerBusy = false;
}

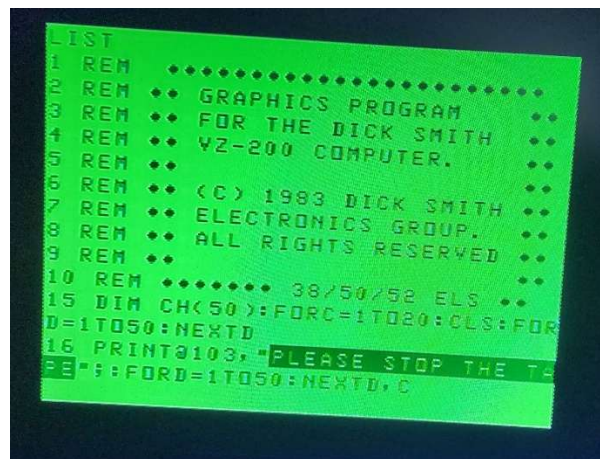
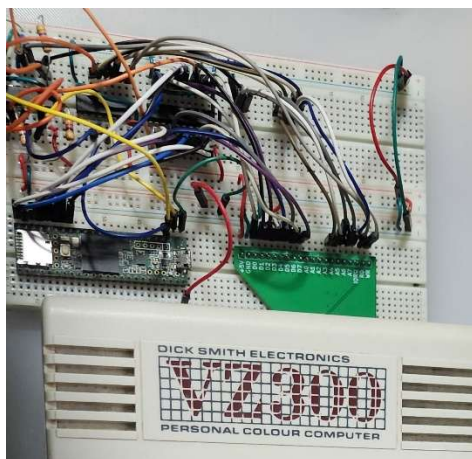
/*
 *
 * ensure all print data is physically written to SD card within 5 seconds
 * in case card is removed or VZ is switched off. A flush() means the file
 * doesn't have to be closed
 *
 */

if (millis() - lastFileBufferWrite >= 5000)
{
    printerFile.flush();
    lastFileBufferWrite = millis();
}
}

```

Results

Below is the breadboard prototype connected to a VZ300, and the output of a LLIST captured to SD card. Standard characters print fine, but the inverse character set is not recognized and prints garbage. An enhancement to the arduino sketch could be to recognize inverse and block graphics characters, and normalise them back to a standard character.




```

1 REM *****
2 REM ** GRAPHICS PROGRAM **
3 REM ** FOR THE DICK SMITH **
4 REM ** VZ-200 COMPUTER. **
5 REM **
6 REM ** (C) 1983 DICK SMITH **
7 REM ** ELECTRONICS GROUP. **
8 REM ** ALL RIGHTS RESERVED **
9 REM **
10 REM ***** 38/50/52 ELS **
11
12 DIM CH(50):FORC=1TO20:CLS:FORD=1TO50:NEXTD
13
14 PRINT@103,"
15 AYYYA
16 Y";:FORD=1TO50:NEXTD,C
17 CLS
18
19 PRINT@97,"GRAPHICS DEMONSTRATION FOR THE"
20
21 PRINT@167,"DICK SMITH
22 COLOUR COMPUTER"
23
24 PRINT@391,"HIT THE SPACE BAR"
25
26 FORI=1TO50
27 A=RND(511)
28 CH(I)=A
29 POKE28672+A,RND(128)+127:NEXTI

```

What next?

The next enhancement will be to add a dip switch to the circuit allowing to switch between plain text output to the SD card, or generate a bitmap file with a “dot matrix” font that will recognize the VZ/6847 character set. I also intend to build a pcb and 3D printed enclosure